

Module 01: PHP Fundamentals for Laravel

Duration: 5-7 days | **Level:** Beginner | **Prerequisites:** Module 00

Learning Objectives

- Write modern PHP 8.2+ code used throughout Laravel
 - Understand OOP concepts: classes, inheritance, interfaces, traits
 - Use namespaces and autoloading
 - Work with arrays, collections mindset, and type hints
-

1. Modern PHP Syntax

Variables & Types

```
<?php

declare(strict_types=1); // Enforce type declarations

$name = 'Alice';          // string
$age = 30;                 // int
$price = 19.99;           // float
$active = true;           // bool
$items = ['a', 'b', 'c']; // array
$nothing = null;

// PHP 8+ union types
function process(int|string $value): void
{
    echo $value;
}
```

Null Safe & Match (PHP 8+)

```
// Nullsafe operator
$country = $user?->address?->country;

// Match expression (better than switch)
$status = match ($task->priority) {
    'low' => 'info',
    'medium' => 'warning',
    'high', 'urgent' => 'danger',
    default => 'secondary',
};
```

Named Arguments

```
Task::create(
    title: 'Fix bug',
    project_id: 1,
    due_at: now()->addDays(3),
);
```

Constructor Property Promotion (PHP 8+)

Laravel uses this everywhere:

```
class TaskController
{
```

```
public function __construct(
private TaskService $taskService,
) {}
}
```

2. Object-Oriented Programming

Classes & Objects

```
<?php

namespace App\Services;

class TaskService
{
    public function __construct(
        private TaskRepository $repository,
    ) {}

    public function complete(Task $task): Task
    {
        $task->status = 'completed';
        $task->completed_at = now();

        return $this->repository->save($task);
    }
}
```

Inheritance & Abstract Classes

```
abstract class BaseController
{
    protected function success(string $message): array
    {
        return ['success' => true, 'message' => $message];
    }
}

class TaskController extends BaseController
{
    // ...
}
```

Interfaces

```
interface Notifiable
{
    public function sendNotification(string $message): void;
}

class EmailNotifier implements Notifiable
{
    public function sendNotification(string $message): void
    {
        Mail::to($this->user)->send(new GenericMail($message));
    }
}
```

Traits (Horizontal Reuse)

Laravel models use traits heavily:

```
trait HasTasks
```

```

{
public function tasks(): HasMany
{
return $this->hasMany(Task::class);
}
}

class User extends Authenticatable
{
use HasFactory, Notifiable, HasTasks;
}

```

Enums (PHP 8.1+)

```

enum TaskStatus: string
{
case Pending = 'pending';
case InProgress = 'in_progress';
case Completed = 'completed';

public function label(): string
{
return match ($this) {
self::Pending => 'Pending',
self::InProgress => 'In Progress',
self::Completed => 'Completed',
};
}
}

// Usage in model
protected function casts(): array
{
return ['status' => TaskStatus::class];
}

```

3. Namespaces & Autoloading

```

<?php
// File: app/Models/Task.php

namespace App\Models;

use App\Enums\TaskStatus;
use Illuminate\Database\Eloquent\Model;

class Task extends Model
{
// ...
}

Composer PSR-4 autoloading maps App\ -> app/:

"autoload": {
"psr-4": {
"App\\": "app/",
"Database\\Factories\\": "database/factories/",
"Database\\Seeders\\": "database/seeders/"
}
}

```

After adding new classes: `composer dump-autoload`

4. Arrays & Laravel Collections

PHP arrays are used everywhere, but Laravel provides **Collections** - fluent, chainable array operations:

```
$tasks = Task::all();

$overdue = $tasks
->filter(fn ($task) => $task->due_at?->isPast())
->sortBy('due_at')
->map(fn ($task) => $task->title)
->values();
```

Common Collection methods: map, filter, reduce, pluck, groupBy, first, each.

5. Error Handling

```
try {
    $task = Task::findOrFail($id);
} catch (ModelNotFoundException $e) {
    abort(404, 'Task not found');
} catch (Throwable $e) {
    report($e); // Log the error
    throw $e;
}
```

Laravel's `abort()` helper throws HTTP exceptions:

```
abort(403, 'Unauthorized');
abort_if($user->cannot('update', $task), 403);
```

6. Closures & Arrow Functions

```
// Closure
$callback = function (Task $task) use ($user) {
    return $task->user_id === $user->id;
};

// Arrow function (short, single expression)
$mine = $tasks->filter(fn ($t) => $t->user_id === $user->id);
```

Laravel routes use closures for simple endpoints:

```
Route::get('/health', fn () => response()->json(['ok' => true]));
```

7. Reading PHP in Laravel Codebase

When reading Laravel source, expect:

Pattern	Example
Facades	Cache::get('key') - static proxy to service
Dependency injection	Constructor/method type-hinted params
Magic methods	__get, __call on Eloquent models
Attributes	#[ObservedBy([TaskObserver::class])]
Return type declarations	: JsonResponse, : void

8. Hands-On Exercises

1. Create app/Enums/TaskPriority.php with cases: Low, Medium, High, Urgent.
 2. Add a label() method returning human-readable strings.
 3. In Tinker: TaskPriority::High->label() should return "High".
 4. Write a function countByStatus(Collection \$tasks): array that returns ['pending' => 5, ...].
-

9. Self-Check Quiz

1. What does declare(strict_types=1) do?
 2. Difference between implements and extends?
 3. What is a trait used for?
 4. How does PSR-4 autoloading find App\Models\Task?
 5. What is the nullsafe operator?
-

Next Module

-> [Module 02: Laravel Basics & Request Lifecycle](#)